This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

Dynamic and Heterogeneous Network Slicing for Vehicular Edge Computing Based on Two-Timescale Reinforcement Learning

Xulong Li, *Student Member, IEEE*, Wencan Mao, *Member, IEEE*, Xinyi Xu, *Member, IEEE*, Yaxi Liu, *Member, IEEE*, Wei Huangfu, *Member, IEEE*, Keping Long, *Senior Member, IEEE*, Yu Xiao, *Member, IEEE*, and Yusheng Ji, *Fellow, IEEE*

***Abstract*—Vehicular Edge Computing (VEC) is an essential part of the Internet of Vehicles (IoV) due to its low latency by moving the computational resources close to the edge. Although the introduction of network slicing into VEC improves resource utilization through dynamic resource allocation based on real-time demands and priorities, it increases the deployment and operational costs. In view of this, this paper envisions a resource allocation strategy for VEC based on network slicing technique, in which the tasks involved are not only dynamic but also heterogeneous. To minimize the system cost (including resource consumption and computation, network slice maintenance and reconfiguration costs), this paper proposes CST-RL, a confidence-based self-adjusting two-timescale reinforcement learning algorithm. This solution performs resource allocation and activation scheduling for network slices on a large timescale, while allocating slices to heterogeneous tasks on a short timescale to meet dynamic demands. In addition, we innovatively utilize critic in reinforcement learning to predict and compare the expected benefits of network slices with versus without reconfiguration. We introduce the Random Network Distillation (RND) technique to assess the confidence level of these benefits, thus providing guidance for network slices to automatically decide whether and when to undergo reconfiguration. Finally, we demonstrate the effectiveness and superiority of CST-RL through simulations. Results show that CST-RL yields 27.77% lower system cost compared to the scheme without network slicing and 15.15% lower system cost compared to performing constant network slicing configuration, with guaranteed Quality-of-Service.**

***Index Terms*—Vehicle edge computing (VEC), network slicing, reinforcement learning (RL), two-timescale optimization, resource allocation.**

I. Introduction

Driven by wireless communication technology and artificial intelligence, the Internet of Vehicles (IoV) is accelerating revolution of the traditional transportation system [1], [2]. The IoV not only helps improve road safety and reduce traffic congestion by enabling assisted and autonomous driving, but also provides drivers with more convenient and personalized services such as onboard infotainment and fatigue monotoring.

To fulfil this demand, Vehicular Edge Computing (VEC) has emerged as a new computing paradigm, shifting computation from the remote cloud to the edge of the network to reduce latency [2]–[5]. Vehicles can also offload computational tasks to the edge computing nodes to reduce computational costs and shorten processing latency. Resource allocation problem for VEC decides how to allocate computational and communication resources from the edge to tasks generated by the vehicles. However, due to the diversity of quality-of-service (QoS) requirements among tasks, the traditional one-size-fits-all resource allocation strategies become inefficient [6], [7]. This necessitates new solutions that dynamically optimize resource allocation to meet varying demands [8].

Network slicing, which allows for service differentiation by dividing the physical infrastructure into multiple logically isolated slices with customized configurations of computing and networking resources, has shown great potential in solving this challenge [9], [10]. Through network slicing technology, vehicles can obtain personalized network services according to their own needs, thus improving their user experience and QoS. However, it also poses new challenges, especially in terms of controlling the cost of deploying network slices and managing the complexity [11]. Specifically, frequent network slicing reconfiguration is often required to accurately match dynamic resource demands, but this significantly increases the reconfiguration overhead of computing and communication resources. Therefore, how to reduce the reconfiguration frequency and optimize the long-term cost of the system while guaranteeing the flexibility of slicing has become a key issue for the implementation of network slicing technology in VEC.

In view of this, this paper focuses on the dynamic and heterogeneous resource allocation problem for VEC and proposes a dynamic network slicing strategy based on Reinforcement Learning (RL), aiming to effectively minimize the overall system cost, including resource consumption and computation, network slice maintenance and reconfiguration costs. Furthermore, a confidence-based self-adjusting two-timescale RL algorithm, referred to as CST-RL, is proposed to decide whether and when to undergo reconfiguration of network slices automatically.

This work was supported in part by the National Key Research and Development Program (Grant No. 2023YFB2903901), in part by the National Natural Science Foundation of China (Grant No. 62301028), the JSPS KAKENHI (Grant No. JP24K02937), and the JST ASPIRE (Grant No. JPMJAP2325). *(Corresponding author: Wei Huangfu.)*

Xulong Li, Xinyi Xu, Yaxi Liu, Wei Huangfu, and Keping Long are with the School of Computer and Communication Engineering, University of Science and Technology Beijing, Beijing 100083, China (e-mail: D202210383@xs.ustb.edu.cn; xinyixu@ustb.edu.cn; yaxi.ustb@gmail.com; huangfuwei@ustb.edu.cn; longkeping@ustb.edu.cn;).

Wencan Mao and Yusheng Ji are with the Information Systems Architecture Science Research Division, National Institute of Informatics, Tokyo 101-8430, Japan. (e-mail: wencan_mao@nii.ac.jp; kei@nii.ac.jp).

Yu Xiao is with the Department of Information and Communications Engineering, Aalto University, Espoo 02150, Finland. (e-mail: yu.xiao@aalto.fi).

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

The contributions of this work are listed as follows:

1) In this paper, we formulate the resource allocation problem for VEC based on the network slicing technique, aiming to provide differentiated vehicular services with minimized system cost.
2) Since the dynamic characteristics of the considered VEC system are reflected in different time scales, we construct the above problem as a two-timescale optimization model. The first part of the model performs resource allocation and activation scheduling for network slices on a large timescale, while the second part of the model allocates slices to heterogeneous tasks on a short timescale to meet dynamic demands.
3) To address the above two-timescale optimization problem, this paper proposes CST-RL, a confidence-based self-adjusting two-timescale reinforcement learning algorithm. Compared with the existing two-timescale optimization schemes with fixed time window size for network slicing, CST-RL proactively adjusts the time window size by predicting the benefits of network slice reconfiguration versus no reconfiguration and their confidence levels, which significantly reduces the system cost.
4) Using simulation, we compare CST-RL with the state-of-the-art baselines. Results show that CST-RL saves 27.77% of system cost compared to the scheme without network slicing and reduces 15.15% of system cost compared to performing constant network slicing configuration, with comparable or higher QoS.

The rest of the paper is organized as follows. Section II reviews the related works. Section III describes the system model and formulates the optimization problem. The proposed CST-RL scheme is detailed in Section IV. Simulation experiments and analysis are carried out in Section V, and finally, conclusions are drawn in Section VI.

II. Related Works

This section reviews the state-of-the-art resource allocation methods in the context of VEC.

A. Resource Allocation for Vehicular Edge Computing

Resource Allocation for VEC has attracted extensive research attention in recent years. Lyapunov optimization and Deep RL (DRL) are commonly used methods to cope with dynamic changes in VEC and optimize the long-term performance of the system. Geng et al. [12] and Huang et al. [13] proposed Lyapunov-based online dynamic resource allocation and computational offloading schemes for VEC systems that addressed the high dynamics of task arrivals and varying wireless network conditions that may severely affect the quality of computational offloading of latency-aware and high-reliability services. Han et al. [14] and Li et al. [15] proposed game theory-based and DRL schemes for the computational resource allocation and dynamic resource allocation problems of Unmanned Aerial Vehicle (UAV)-assisted VEC systems, respectively. Zhao et al. [16] developed a computational offloading model based on VEC considering the data dependency of the tasks, and developed a DRL-based mobile sensing dependent resource allocation scheme for urban VEC to minimize the average response time and average energy consumption of the system. Similarly, Zhao et al. [17] and Hu et al. [18] used DRL to optimize computational offloading and resource allocation strategies for VEC systems. However, the high mobility of vehicles and the spatial and temporal heterogeneity of computing tasks make efficient resource allocation a serious challenge. How to achieve precise resource allocation for heterogeneous computing tasks and differentiated QoS demands in dynamic environments has become a great challenge for VEC.

To solve the above problems, researchers have introduced network slicing technology into VEC. Network slicing, through logical isolation and on-demand customization, enables the construction of multiple virtualized networks on the same physical infrastructure to provide differentiated services for heterogeneous vehicular tasks. To address the heterogeneous computing resource allocation problem in VEC considering the mobility of vehicles, several research [19]–[22] proposed frameworks for edge computing resource allocation based on network slicing techniques. These literatures design optimization strategies based on heuristic algorithms and DRL, aiming to provide differentiated QoS guarantees for different types of computing tasks. In addition, Khan et al. [23] provided an in-depth study of network slicing techniques at the radio access network level, especially in the application of downlink resource allocation. The study aims to support two different vehicle service demands, namely autonomous driving and infotainment, through refined slicing design. However, in order to accurately match dynamic resource requirements, network slicing often requires frequent reconstruction, which significantly increases the reconstruction overhead of computing and communication resources. Therefore, how to reduce the reconstruction frequency and optimize the long-term cost of the system while ensuring the flexibility of slicing has become a key issue in the implementation of network slicing technology in VEC.

B. Resource Allocation for Two-Timescale Network Slicing

To this end, the network slicing management and resource allocation based on the two-timescale optimization framework is a promising solution that has attracted attention. Li et al. [24] and Feng et al. [25] proposed Lyapunov-based two-timescale optimization schemes with the objectives of maximizing the long-term average network utility and minimizing the resource allocation latency of the VEC system, respectively. Fang et al. [26] and Huang et al. [27] proposed a two-timescale network slicing and resource allocation optimization framework for satellite edge computing networks, in which resource allocation strategies are learned based on RL algorithms on short timescales, while heuristic-based algorithms optimize resource slicing on long timescales. Similarly, Han et al. also proposed a two-timescale stochastic optimization scheme for the problem of minimizing resource allocation latency in satellite edge computing networks in [28]. Zhang et al. [29] proposed a two-timescale resource management scheme for

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

TABLE I
MAIN RELATED WORK SUMMARY

Ref.	Main Contributions: Scenario	Main Contributions: Solution Method	Features: Type of Allocated Resources	Features: Optimization Objective	Two Timescale	Network Slicing
[12]	Cloud-Edge Collaboration in VEC Network	Lyapunov optimization	Spectrum, computing	Minimize average latency	✘	✘
[13]	NOMA-based VEC Network	Lyapunov optimization	Power, channel	Maximize transmission reliability	✘	✘
[14]	UAV-assisted VEC network	Game theory, DRL	Channel, Bandwidth	Minimize Age of Information	✘	✘
[15]	UAV-assisted VEC network	Game theory, DRL	Spectrum, computing	Minimize training delay and energy	✘	✘
[16]	Urban VEC networks	DRL	Spectrum, computing	Minimize response time and energy	✘	✘
[17]	VEC network	DRL Kuhn-Munkres Matching	Spectrum, computing	Minimize system latency	✘	✘
[18]	NOMA-based VEC network	Graph convolutional DRL	Power, computing	Maximize system utility	✘	✘
[19]	Slicing-Based for VEC Network	Heuristic algorithm	Spectrum, computing	Maximize offloaded task volume	✘	✔
[20]	Autonomous VEC network	Multi-agent DQN	Spectrum, computing	Maximize resource utilization	✔	✔
[21]	Space-air-ground VEC Network	double DQN	Spectrum, computing	Maximize the average number of completed tasks	✔	✔
[22]	Slicing-Based VEC Network	Heuristic algorithm	Storage, computing	Mmaximize the number of satisfied offloading demands	✘	✔
[23]	Ultra-reliable VEC Network	Clustering algorithm	Spectrum, computing	Minimize transmission delay	✘	✔
[24]	UAV-assisted VEC	Two-timescale Lyapunov optimization	Storage, computing	Maximize the long-term average network utility	✔	✔
[25]	UAV-assisted VEC	Two-timescale Lyapunov optimization	Spectrum, power	Minimize the long-term system cost	✔	✔
[26]	Satellite edge computing network	DRL, greedy algorithm	Spectrum, computing	Minimize latency and energy	✔	✔
[27]	Satellite edge computing network	artificial electric field, DRL	Spectrum, computing	Minimize system cost	✔	✔
[28]	Satellite-terrestrial IoT	Hierarchical DRL	Spectrum, computing	Minimize long-term offloading delay	✔	✔
[29]	Cloud Radio Access Network	Two-stage stochastic programming	Channel, power	Maximize tenant profit	✔	✔
[30]	Edge-cloud collaborative VEC Network	Two-stage DRL	Spectrum, computing	Minimize network slicing cost	✔	✔
[31]	Slicing-Based for VEC Network	Hierarchical Meta-RL	Spectrum	Maximize spectral efficiency	✔	✔
[32]	Slicing-Based for VEC Network	Collaborative MADQN	Spectrum, computing	Minimize long-term energy	✔	✔
[33]	Slicing-Based for VEC Network	MAPPO	Spectrum	Maximize spectral efficiency	✔	✔
our	Slicing-Based for VEC Network	Confidence-based self-adjusting two-timescale RL	Spectrum, computing	Minimize system cost	✔	✔

network slicing in cloud radio access networks, aiming to maximize the profit of the system, i.e., the difference between the revenue from its users and the cost of resource reservation. Wu et al. [30] investigated the network slicing problem for edge-cloud orchestrated in-vehicle networks. In order to reduce the network slicing cost while meeting the quality of service requirements, an RL-based two-timescale network slicing framework is proposed, in which slicing deployment, edge resource allocation, and cloud resource allocation are optimized on a large timescale, and resource allocation and task distribution are optimized on a small timescale. Further, literatures [31]–[33] proposed two-timescale network slicing optimization schemes based on hierarchical meta-RL, Collaborative Multi-agent Deep Q-Network (MADQN), and Multi-agent Proximal Policy Optimization (MAPPO), respectively. Table I shows a summary and comparison of the main related work.

However, the aforementioned two-timescale-based network slicing strategies all preset a fixed network slicing time window, thus fail to adequately incorporate the dynamic characteristics of the network and the stochastic nature of the computational tasks. Specifically, with a small time window size, although the network slices can be reconfigured frequently to accurately adapt to the time-varying resource demands of vehicular computing tasks, such frequent reconfiguration operations significantly increase the overall cost of the system. On the contrary, with a larger time window size, the resource allocation of network slices and the adjustment of their activation states will not be fast enough, making it difficult for the system to effectively cope with the time-varying resource demands of vehicular computing tasks. Motivated by the above, we propose a two-timescale network slicing scheme

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

for VEC, where the time window size dynamic adjusts for network slicing based on DRL.

III. System Model and Problem Formulation

This section details the exemplary scenario, the system model, and the problem formulation.

A. Exemplary Scenario

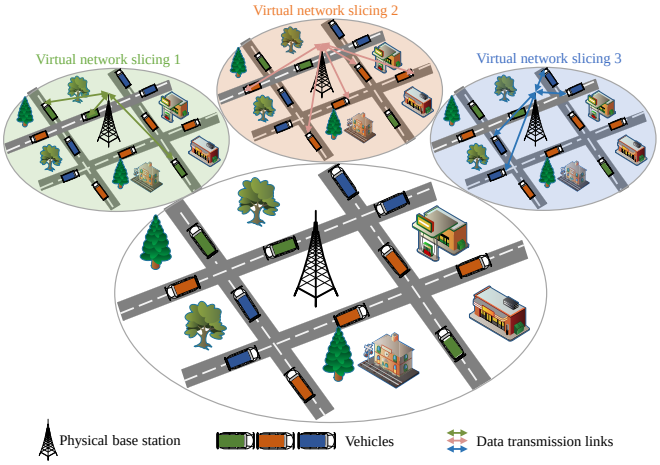


Fig. 1. An exemplary scenario of dynamic resource allocation for VEC.

This paper investigates the dynamic and heterogeneous resource allocation problem based on network slicing techniques. We demonstrate an exemplary scenario of VEC in Fig. 1, which consists of a multi-antenna base station with high-performance edge servers and multiple vehicles. The vehicles need to offload their computational tasks to the edge servers co-located with the base station for processing, and then the base station returns the processing results to the vehicles. In this work, only the network demand relevant to VEC is considered. Let the set of antennas of the base station and vehicles be $\mathcal{L} = \{1, \cdots, L\}$ and $\mathcal{V} = \{1, \cdots, V\}$, respectively. In addition, we utilize network slicing technology to segment the physical network into multiple logically isolated virtual network slices to provide differentiated services for vehicles with heterogeneous computing tasks, taking the mobility of vehicles into account. Following the common setting of existing studies [12], this paper assumes that the base station centrally manages the network slices and allocates bandwidth and computing resources, and ignores the delay caused by resource allocation.

B. Heterogeneous Computational Tasks Models

In this paper, we assume that the computational tasks generated by the vehicles are heterogeneous, and that these different types of tasks vary in terms of computational resource requirements, data size, and sensitivity to latency. Specifically, this paper adopts a discrete-time system, i.e., the entire decision cycle is partitioned into T equal time slots $t \in \mathcal{T} = \{1, ..., T\}$. Let $D_v(t) = \{I_v(t), O_v(t), C_v(t), L_v(t), J_v(t)\}$ be the computational task of the v-th vehicle in the t-th time slot, where I_v and O_v are the data sizes of the original computational task generated by the vehicle and the result of its being processed by the base station, respectively, in units of bits; C_v is the computational resources required to process the computational task, in bit/Cycle; L_v is the maximum latency constraint of the computational task in seconds, and J_v is the type of the computational task. It is worth noting that the computing tasks of vehicles are not only heterogeneous but also the types $J_v(t)$ of their computing tasks are time-varying.

We consider three types of computational tasks: *computational-intensive* (such as autonomous driving environment perception and decision-making reasoning tasks, real-time construction and update tasks of high-precision maps, etc), *data-intensive* (such as in-vehicle multimedia and infotainment tasks, vehicle health monitoring and big data analysis, etc), and *latency-sensitive* (such as real-time collision warning and avoidance tasks, vehicle dynamic control command execution tasks, etc). Assuming $\mathcal{V}_\mathrm{C}$, $\mathcal{V}_\mathrm{D}$, and $\mathcal{V}_\mathrm{L}$ are the sets of vehicles with these types of computational tasks, respectively, then $\mathcal{V}_\mathrm{C} \cup \mathcal{V}_\mathrm{D} \cup \mathcal{V}_\mathrm{L} = \mathcal{V}$.

C. Network Slicing-Enabled Resource Allocation Models

The VEC system based on network slicing technology constructs end-to-end logically isolated heterogeneous resource allocation channels, effectively mitigating computation resource contention conflicts in multi-vehicle scenarios and realizing a Network-as-a-Service (NaaS) differentiated resource provisioning model. This model enables heterogeneous tasks to obtain differentiated network QoS guarantees matched to their characteristics during the offloading process, effectively resolving resource waste and resource allocation performance bottlenecks caused by the "one-size-fits-all" approach of traditional networks.

1) Network Slicing Models: We utilize network slicing techniques to slice the physical network into three logically isolated virtual network slices to provide differentiated services for different types of computing tasks, i.e., *high-computing network slice* $\mathrm{S_C}$ for compute-intensive tasks, *high-bandwidth network slice* $\mathrm{S_D}$ for data-intensive tasks, and *low-latency network slice* $\mathrm{S_L}$ for latency-sensitive tasks. In addition, network slices can dynamically adjust their activation states based on real-time requirements. We define $B(t) = \{b_C(t), b_D(t), b_L(t)\}$ as the activation state of the three network slices and its value is 1 when the network slice is active and 0 otherwise.

Each of the three network slices has its own relatively isolated computational and bandwidth resources. Let F be the total computational resources of the base station, $\alpha_{\mathrm{F,\,C}}, \alpha_{\mathrm{F,\,D}}$, and $\alpha_{\mathrm{F,\,L}}$ are the ratio of computational resources to total computational resources for network slices $\mathrm{S_C}$, $\mathrm{S_D}$, and $\mathrm{S_L}$, respectively. Similarly, let W be the total bandwidth resource of the base station, $\alpha_{\mathrm{W,\,C}}, \alpha_{\mathrm{W,\,D}}$, and $\alpha_{\mathrm{W,\,L}}$ are the ratio of bandwidth resouces for network slices $\mathrm{S_C}$, $\mathrm{S_D}$, and $\mathrm{S_L}$, respectively.

2) Resource Allocation Models: Vehicles utilize computational and bandwidth resources allocated by the corresponding network slices to offload heterogeneous tasks, which can be

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

divided into three phases. In the first phase, all the vehicles use the *orthogonal frequency division multiple access (OFDMA)* technique to upload computational tasks to the base station, which implies that there is no inter-vehicle interference. Therefore, the data rate of the v-th vehicle's uploading computational tasks to the base station at t-th time slots is

$$R_v^{\mathrm{I}}(t) = W_v(t)\log_2\left(1+\frac{\left|\mathbf{H}_v^{\mathrm{I}}(t)\mathbf{w}_v^{\mathrm{I}}(t)\right|^2 p_v^{\mathrm{I}}}{\sigma^2}\right), \tag{1}$$

where p_v^{I} is the transmission power of the v-th vehicle, σ is the Gaussian noise power, $\mathbf{w}_v^{\mathrm{I}} \in \mathbb{C}^{L\times 1}$ is the receiving beamforming vector for the v-th vehicle at the base station, $\mathbf{H}_v^{\mathrm{I}} \in \mathbb{C}^{1\times L}$ is the channel between the v-th vehicle and the base station [34], and W_v is the bandwidth resources allocated by the base station to the v-th vehicle, it can be expressed as

$$W_v(t) = \begin{cases} \alpha_{\mathrm{W,\,C}}(t)\beta_{W,v}(t)W, \forall v \in \mathcal{V}_{\mathrm{C}}(t), \\ \alpha_{\mathrm{W,\,D}}(t)\beta_{W,v}(t)W, \forall v \in \mathcal{V}_{\mathrm{D}}(t), \\ \alpha_{\mathrm{W,\,L}}(t)\beta_{W,v}(t)W, \forall v \in \mathcal{V}_{\mathrm{L}}(t), \end{cases} \tag{2}$$

where $\beta_{W,v}$ is the ratio of the bandwidth resources allocated by the base station to the v-th vehicle to the total available resources of the corresponding network slice. Therefore, the latency of the v-th vehicle to upload the computational task to the base station in the first phase is

$$T_v^{\mathrm{I}}(t) = \frac{I_v(t)}{R_v^{\mathrm{I}}(t)}. \tag{3}$$

In the second phase, the base station receives the computational tasks offloaded by the vehicles and allocates corresponding computational resources for their processing. In the t-th time slot, the latency caused by the base station to process the computational task offloaded by the v-th vehicle is

$$T_{\mathrm{v}}^{\mathrm{II}}(t) = \frac{I_v(t)C_v(t)}{F_v(t)}, \tag{4}$$

where F_v is the computational resources allocated by the base station to the v-th vehicle, it can be expressed as

$$F_v(t) = \begin{cases} \alpha_{\mathrm{F,\,C}}(t)\beta_{F,v}(t)F, \forall v \in \mathcal{V}_{\mathrm{C}}(t), \\ \alpha_{\mathrm{F,\,D}}(t)\beta_{F,v}(t)F, \forall v \in \mathcal{V}_{\mathrm{D}}(t), \\ \alpha_{\mathrm{F,\,L}}(t)\beta_{F,v}(t)F, \forall v \in \mathcal{V}_{\mathrm{L}}(t), \end{cases} \tag{5}$$

where $\beta_{F,v}$ is the ratio of the computational resources allocated by the base station to the v-th vehicle to the total available resources of the corresponding network slice.

In the third phase, the base station sends the processing results of the computational tasks back to the corresponding vehicle. The data rate R_v^{III} of the base station to transmit the processing result of the computation task to the v-th vehicle at t-th time slots is

$$R_v^{\mathrm{III}}(t) = W_v(t)\log_2\left(1+\frac{\left|\mathbf{w}_v^{\mathrm{III}}(t)^H\mathbf{H}_v^{\mathrm{III}}(t)\right|^2 p_v^{\mathrm{III}}}{\sigma^2}\right), \tag{6}$$

where $\mathbf{w}_v^{\mathrm{III}}$ and p_v^{III} are the transmit beamforming vector and power of the base station to the v-th vehicle, respectively. Therefore, in the third stage, the latency for the base station to transmit the processing results of the computational task to the v-th vehicle is

$$T_v^{\mathrm{III}}(t) = \frac{O_v(t)}{R_v^{\mathrm{III}}(t)}. \tag{7}$$

Finally, the total processing latency of the computing task for the v-th vehicle at t-th time slot is the sum of the delays of the three phases, i.e. $T_v(t) = T_v^{\mathrm{I}}(t) + T_v^{\mathrm{II}}(t) + T_v^{\mathrm{III}}(t)$.

Since most tasks in VEC scenarios have significant timeliness, when the processing latency of a computing task exceeds its maximum latency constraint, it loses its value for further processing and will be judged as a processing failure and discarded.

D. System Cost Model

The system cost includes resource consumption cost, network slice maintenance cost, network slice reconfiguration cost, and computational task processing cost. We define the system cost as the weighted sum of these four costs, and it has no unit. Thus, the system cost at t-th time slot can be expressed as

$$\begin{aligned} C(t) &= \sum_{*\in\{\mathrm{C,\,D,\,L}\}} C_*(t) \\ &= \sum_{*\in\{\mathrm{C,\,D,\,L}\}} \omega_{\mathrm{res}} C_{\mathrm{res},\,*}(t) + \omega_{\mathrm{mai}} C_{\mathrm{mai},\,*}(t) \\ &\qquad + \omega_{\mathrm{rec}} C_{\mathrm{rec},\,*}(t) + \omega_{\mathrm{com}} C_{\mathrm{com},\,*}(t). \end{aligned} \tag{8}$$

where ω_{res}, ω_{mai}, ω_{rec}, and ω_{com} are weighting factors.

The resource consumption cost of network slices is the weighted sum of bandwidth resources and computational resources when that network slice is in the active state, denoted as

$$C_{\mathrm{res},\,*}(t) = b_*(t)(\omega_{\mathrm{res,\,W}}\alpha_{\mathrm{W},\,*}(t)W + \omega_{\mathrm{res,\,F}}\alpha_{\mathrm{F},\,*}(t)F), \tag{9}$$

where $\omega_{\mathrm{res,\,W}}$ and $\omega_{\mathrm{res,\,F}}$ are weighting factors.

Maintaining an active network slice incurs costs, such as the costs of hardware equipment wear and tear and system operation energy consumption, the costs of virtualization platform and network function maintenance, as well as the human and technical investment costs for troubleshooting and performance monitoring. Therefore, the network slice maintenance cost is

$$C_{\mathrm{mai},\,*}(t) = b_*(t)P_{\mathrm{mai}}, \tag{10}$$

where P_{mai} is the maintenance cost of network slicing.

The network slice reconfiguration cost is related to the changes in the resource and activation states of the network slice in two consecutive windows. Since the destruction and resource release of network slices is fast and inexpensive, we only consider the cost of network slice generation and resource increase, so the network slice reconfiguration cost can be expressed as

$$\begin{aligned} C_{\mathrm{rec},\,*}(t) =& b_*(t)(\omega_{\mathrm{rec,\,W}}W\left[\alpha_{\mathrm{W},\,*}(t) - \alpha_{\mathrm{W},\,*}(t-1)\right]^+ \\ &+ \omega_{\mathrm{rec,\,F}}F\left[\alpha_{\mathrm{F},\,*}(t) - \alpha_{\mathrm{F},\,*}(t-1)\right]^+) \\ &+ \omega_{\mathrm{rec,\,G}}\left[b_*(t) - b_*(t-1)\right]^+, \end{aligned} \tag{11}$$

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

where $\omega_{\text{rec, W}}$, $\omega_{\text{res, F}}$, and $\omega_{\text{res, G}}$ are weighting factors.

The computational task processing cost is related to the penalties caused by processing latency of the computational tasks of the vehicles belonging to the network slice. When the processing latency of a computing task does not exceed its maximum latency constraint, the task is processed successfully, and the cost is the processing latency. Otherwise, the task processing fails and its cost is P_{com}. Therefore, the computational task processing cost can be expressed as

$$C_{\text{com, }*}(t) = \sum_{v \in \mathcal{V}_*(t)} \begin{cases} T_v(t), T_v(t) \leq L_v(t), \\ P_{\text{com}}, \text{otherwise.} \end{cases} \tag{12}$$

E. Problem Formulation

Since the system uses OFDMA technique to eliminate the inter-user interference, so we assume that when the base station receives the computational tasks sent by vehicles, it adopts Maximum Ratio Combining (MRC) to design the receive beamforming matrix $\mathbf{w}_v^{\text{I}}$. By applying weights proportional to the conjugate of the channel to the received signal, MRC causes the useful signal components to add up in-phase while the noise components add up randomly, thereby maximizing the received signal-to-noise ratio for each vehicle. Consequently, the receive beamforming matrix is $\mathbf{w}_v^{\text{I}}(t) = \frac{\mathbf{H}_v^{\text{I}}(t)^H}{||\mathbf{H}_v^{\text{I}}(t)||}$.

Similarly, we assume that when the base station returns the processed results of computational tasks to vehicles, it employs Maximum Ratio Transmission (MRT) to design the transmit beamforming matrix $\mathbf{w}_v^{\text{III}}$. By applying weights proportional to the conjugate of the channel to the transmitted signal, MRT causes the signals to add up in-phase at the user end, thereby enhancing the received signal strength. Thus, the transmit beamforming matrix is $\mathbf{w}_v^{\text{III}}(t) = \frac{\mathbf{H}_v^{\text{III}}(t)^H}{||\mathbf{H}_v^{\text{III}}(t)||}$.

The goal of this paper is to minimizing the system cost by jointly optimizing the activation state of the network slices $\boldsymbol{B} = \{B(t), \forall t \in \mathcal{T}\} = \{b_C(t), b_D(t), b_L(t), \forall t \in \mathcal{T}\}$, the resource allocation vector for the network slice level $\boldsymbol{\alpha} = \{\alpha(t), \forall t \in \mathcal{T}\} = \{\alpha_{\text{W, C}}(t), \alpha_{\text{W, D}}(t), \alpha_{\text{W, L}}(t), \alpha_{\text{F, C}}(t), \alpha_{\text{F, D}}(t), \alpha_{\text{F, L}}(t), \forall t \in \mathcal{T}\}$, and the resource allocation vector for the vehicle-level $\boldsymbol{\beta} = \{\beta(t), \forall t \in \mathcal{T}\} = \{\beta_{F,v}(t), \beta_{W,v}(t), \forall v \in \mathcal{V}, \forall t \in \mathcal{T}\}$ at the base station. Therefore, the optimization problem can be formulated as

$$\text{P1: } \min_{\boldsymbol{B}, \boldsymbol{\alpha}, \boldsymbol{\beta}} \sum_{t=0}^{T} C(t; B(t), \alpha(t), \beta(t), w^{\text{I}}(t), w^{\text{III}}(t)) \tag{13}$$

$$\text{s.t. } b_*(t) \in \{0, 1\}, \forall * \in \{\text{C,D,L}\}, \forall t \in \mathcal{T}, \tag{13a}$$

$$\sum_{* \in \{\text{C, D, L}\}} b_*(t) \alpha_{\text{W, }*}(t) \leq 1, \forall t \in \mathcal{T}, \tag{13b}$$

$$\sum_{* \in \{\text{C, D, L}\}} b_*(t) \alpha_{\text{F, }*}(t) \leq 1, \forall t \in \mathcal{T}, \tag{13c}$$

$$\sum_{v \in \mathcal{V}_*(t)} \beta_{W,v}(t) = 1, \forall t \in \mathcal{T}, \forall * \in \{\text{C, D, L}\}, \tag{13d}$$

$$\sum_{v \in \mathcal{V}_*(t)} \beta_{F,v}(t) = 1, \forall t \in \mathcal{T}, \forall * \in \{\text{C, D, L}\}, \tag{13e}$$

Constraints 13(b) and 13(c) indicate that the sum of the bandwidth and computational resources of the three network slices does not exceed the total resources of the system, and constraints 13(d) and 13(e) indicate that the sum of the bandwidth and computational resources of the vehicles within each slice is equal to the total resources of its corresponding network slice, respectively.

Theorem 1: The formulated resource allocation problem P1 is NP-hard.

Proof: We establish the NP-hardness of Problem P1 through a polynomial-time reduction from the classic 0-1 Knapsack Problem. To demonstrate this, consider a simplified instance of Problem P1 restricted to a single time slot, focusing specifically on the binary slice activation decision variables b_* while fixing the continuous resource allocation variables to a set of pre-determined feasible configurations. We further assume, without loss of generality, that computational resources are abundant, thereby rendering the resource limitation dominated solely by the bandwidth constraint. In this setting, the optimization reduces to selecting a subset of network slices to activate subject to a single resource capacity, a structure that maps directly to the 0-1 Knapsack Problem.

Specifically, the candidate network slices act as the items, where the bandwidth resource ratio $\alpha_{\text{W},*}$ required by each slice corresponds to the item's weight, and the normalized total bandwidth capacity represents the knapsack capacity. The "value" v_* of each item is defined as the net cost saving for the system, implying that maximizing the total value is mathematically equivalent to minimizing the total system cost, with the binary variable b_* representing the decision to include the item. Consequently, the sub-problem seeks to maximize $\sum v_* b_*$ subject to the capacity constraint $\sum_{* \in \{\text{C, D, L}\}} b_* \alpha_{\text{W, }*} \leq 1$, which is the exact formulation of the 0-1 Knapsack Problem. Since this standard problem is known to be NP-hard and represents a special case embedded within our general problem P1, it follows that Problem P1 is strictly **NP-hard**. ■

IV. Proposed CST-RL Scheme

In this section, the optimization problem is first reconstructed as a two-timescale Markov Decision Process (MDP), and then the proposed CST-RL scheme is presented.

A. Two Timescale Optimization

In order to cope with the diversity of vehicular computing tasks more effectively, the resource allocation of network slices needs to be adjusted in real time to accurately match the resource demand. However, frequent network slice reconfiguration operations, including the adjustment of resource allocation and the dynamic generation of network slices, will substantially increase the system cost. Therefore, we transform the original optimization problem P1 into a two-timescale optimization framework including *network slice-level* and *vehicle-level* subproblems at different time scales, respectively.

Specifically, given the relatively slow change in the types of vehicular computing tasks, the resource requirements of network slices also show a slower changing trend. Therefore,

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

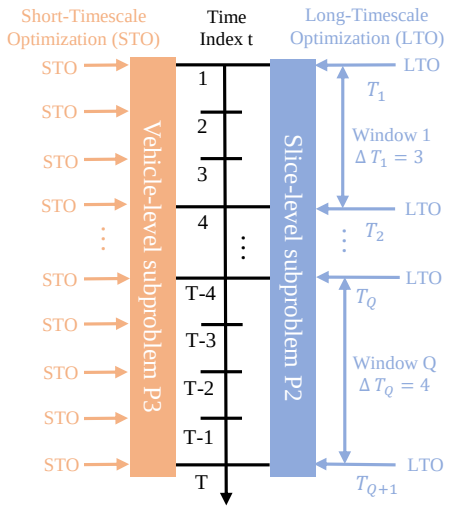


Fig. 2. The two-timescale optimization framework.

in order to reduce the system cost, we focus on optimizing the resource allocation variables of the network slices and their activation states (i.e., the slice-level optimization subproblem) on a long time scale. As shown in Fig. 2, the network slices are reconstructed only when the time slot $t \in \mathcal{T}_Q = \{T_1, T_2, \cdots, T_Q\}$. That is, there are a total of Q network slicing time windows throughout the decision period, where the q-th time window starts from the T_q-th time slot and lasts until the T_{q+1}-th time slot, with a size of $\Delta T_q = T_{q+1} - T_q$. Therefore, the optimization subproblem for long-timescale can be formulated as

$$\text{P2:} \min_{\boldsymbol{B},\boldsymbol{\alpha}} \sum_{q=1}^{Q} \sum_{t=T_q}^{T_{q+1}} C(t; B(t), \alpha(t) | \beta(t)) \tag{31}$$

$$\text{s.t.} \quad (13a) - (13c), \tag{14a}$$
$$B(t) = B(T_q), \forall t \in [T_q, T_{q+1}), \tag{14b}$$
$$\alpha(t) = \alpha(T_q), \forall t \in [T_q, T_{q+1}), \tag{14c}$$

where the constraints (14b) and (14c) show that the activation states and the resource allocation variables of the network slices are reconfigured at the beginning of each time window and remain constant throughout the time window.

In contrast, the relatively fast changes in the vehicle positions and wireless channel states require us to respond quickly on a short time scale by optimizing the resource allocation variables of the vehicles, (i.e., the vehicle-level optimization subproblem). As shown in Fig. 2, the short-timescale optimization decisions are made in each time slot to precisely adjust the vehicle's computing resources and bandwidth resource allocation, thereby accurately matching the vehicle's resource requirements. Therefore, the optimization subproblem for long-timescale can be formulaic as:

$$\text{P3:} \min_{\boldsymbol{\beta}} \sum_{q=1}^{Q} \sum_{t=T_q}^{T_{q+1}} C(t; \beta(t) | B(t), \alpha(t)) \tag{15}$$

$$\text{s.t.} \quad (13d) - (13e). \tag{15a}$$

Consequently, within the two-timescale optimization framework, the slice-level optimization subproblem is optimized on a longer timescale at a lower frequency (e.g., refreshing slice configuration every several time slots). Conversely, the vehicle-level optimization subproblem operates on a shorter timescale at a higher frequency (being optimized per time slot). Thus, the control action per time slot involves two cases:

1) When $t \in \mathcal{T}_Q$ (network slicing reconstruction): First, execute long-timescale slice-level optimization to reconfigure network slices, updating slice resource allocation and activation status. Then, execute short-timescale vehicle-level optimization to optimize vehicle resource allocation variables.
2) When $t \notin \mathcal{T}_Q$ (network slicing non-reconstruction): Execute only the short-timescale vehicle-level optimization (optimizing vehicle resource allocation based on the current slice allocation and activation status).

This two-timescale optimization framework effectively balances system cost and demand matching accuracy. It not only avoids the excessive system overhead inherent in the frequent reconfiguration of network slices, but also achieves more precise matching of vehicle resource demands.

B. Problem Reconstruction in Two-Timescale MDP

In order to reduce the system cost and realize automatic generation and destruction of network slices and automatic adjustment of resource allocation, we model the above problem as a two-timescale MDP, which can be represented as a tuple including six elements $\{\mathbb{S}, \mathbb{A}_S, \mathbb{A}_L, \mathbb{R}, \mathbb{P}, \gamma\}$, where $\mathbb{S}$ is the state space of the system including information of the last time resource allocation variables at the slice level and vehicle level of the network, the current time slot channel state, and the information of the computational tasks of the vehicles, so the state $s(t) \in \mathbb{S}$ of the t-th time slot can be denoted as

$$\begin{aligned} s(t) = \{\{&\mathbf{H}_v^{\mathrm{I}}(t), \mathbf{H}_v^{\mathrm{III}}(t), I_v(t), O_v(t), C_v(t), L_v(t), J_v(t), \\ &\beta_{F,v}(t-1), \beta_{W,v}(t-1), \forall v \in \mathcal{V}\}, \{b_*(t-1), \\ &\alpha_{\mathrm{F},*}(t-1), \alpha_{\mathrm{W},*}(t-1), \forall * \in \{C, D, L\}\}. \end{aligned} \tag{16}$$

$\mathbb{A}_L$ and $\mathbb{A}_S$ are the two parts of the system's action spaces: the one at the network slice-level of the long timescale, and the other at the vehicle-level of the short time scale, respectively. The slice-level actions in the long time scale include the resource allocation variables to the network slices and the dynamic activation states of the network slices, so the long timescale action $a_L(t) \in \mathbb{A}_L$ of the t-th time slot can be denoted as

$$a_L(t) = \{b_*(t), \alpha_{\mathrm{F},*}(t), \alpha_{\mathrm{W},*}(t), \forall * \in \{C, D, L\}\}. \tag{17}$$

The action space at the vehicle-level of the short time scale includes the resource allocation variables to the vehicles, so the short timescale action $a_S(t) \in \mathbb{A}_S$ of the t-th time slot can be denoted as

$$a_S(t) = \{\{\beta_{F,v}(t), \beta_{W,v}(t), \forall v \in \mathcal{V}\}\}. \tag{18}$$

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

$\mathbb{R}$ is the reward function of the system. The reward function $r(t) \in \mathbb{R}$ of the t-th time slot can be denoted as

$$r(t) = -C(t; B(t)) = -\sum_{*\in\{\text{C, D, L}\}} C_*(t). \tag{19}$$

$\mathbb{P}$ is the transition dynamics of the system, the transition dynamics $p(t) \in \mathbb{P}$ of the t-th time slot can be expressed as $p(t) = \Pr(s(t+1), r(t)|s(t), a_L(t), a_S(t))$; $\gamma \in [0, 1]$ is the discount factor, which is used to balance the importance of immediate and future rewards when updating the strategy.

C. Self-Adjusting Two-Timescales RL

For the above two-timescale MDPs, we propose in this section CST-RL, which updates the policy π by interacting with the environment with the goal of maximizing the long-term cumulative reward. The policy $\pi = \{\pi_L, \pi_S\}$ consists of two parts, where π_L is the policy for the long timescale optimization subproblem P2, which is the mapping of the system state space $\mathbb{S}$ to the long timescale actions space $\mathbb{A}_L$ (i.e., $\pi_L := \mathbb{S} \to \mathbb{A}_L$), which is parameterized as $\pi_L^{\theta_L}(s)$; π_S is the policy for the short time scale optimization subproblem P3, which is the mapping of the system state space $\mathbb{S}$ and the long timescale actions space $\mathbb{A}_L$ to the short timescale actions space $\mathbb{A}_S$ (i.e., $\pi_S := \mathbb{S} \cup \mathbb{A}_L \to \mathbb{A}_S$), which is parameterized as $\pi_S^{\theta_S}(s, a_L)$.

In addition, CST-RL is embedded with a specially designed critic mechanism, which is used to evaluate the expected cumulative reward to be obtained by following the current strategy π by taking action a in a given state s, i.e., the state-action value (Q-value) function, which can be represented as

$$Q_\pi(s, a) = \mathbb{E}_{a(t:T)\sim\pi}\left[\sum_{t'=t}^{T} \gamma^{t'-t} r(t') \Big| s(t) = s, a(t) = a\right] \tag{20}$$

Overestimation of the Q value is a common problem in RL. CST-RL is based on the Actor-Critic framework, and the problem of overestimation of its Q value is further worsened due to the coupling of the policy network and the value function network. In CST-RL, there are *double-critic* to mitigate the problem of overestimation and *deep neural networks (DNNs)* is used to parameterize them, i.e., $Q_\pi^{\varphi_1}$ and $Q_\pi^{\varphi_2}$. Further, CST-RL also uses a slowly updating target network to stabilize the training, where the target networks of the policy network and the two critic networks are parameterized as $\pi_L^{\theta_L'}$, $\pi_S^{\theta_S'}$, $Q_\pi^{\varphi_1'}$ and $Q_\pi^{\varphi_2'}$, respectively.

The critic network updates its parameters by minimizing the temporal difference loss, i.e.,

$$L(\varphi_i) = (Q_\pi^{\varphi_i'}(s, a) - y)^2 . i = 1, 2, \tag{21}$$

where y is the target value, which can be expressed as follows:

$$y = r + \gamma \min_{i=1,2} Q_\pi^{\varphi_i'}(s', \tilde{a}'), \tag{22}$$

where $\tilde{a}' = \{\tilde{a}_L', \tilde{a}_S'\} = \{\pi_L^{\theta_L'}(s'), \pi_L^{\theta_L'}(s', \pi_L^{\theta_L'}(s'))\}$.

The policy network updates its parameters by maximizing the state-action value (i.e., Q-value) of

$$J(\theta_L, \theta_S) = \min_{i=1,2} Q_\pi^{\varphi_i}(s, \pi_L^{\theta_L}(s), \pi_S^{\theta_S}(s, \pi_L^{\theta_L}(s))). \tag{23}$$

To enhance the stability and convergence of the algorithm, the short-timescale policy network is assigned a higher learning rate than the long-timescale policy network.

The target network is updated by soft updates, i.e.,

$$\begin{aligned} \theta_L' &= \iota\theta_L + (1-\iota)\theta_L', \\ \theta_S' &= \iota\theta_S + (1-\iota)\theta_S', \\ \varphi_i' &= \iota\varphi_i + (1-\iota)\varphi_i', i = 1, 2, \end{aligned} \tag{24}$$

where ι is the soft update rate.

D. Random Network Distillation

Random Network Distillation (RND) is an exploration technique used in RL, specifically designed to encourage agents to explore novel states in environments where extrinsic rewards are sparse or delayed [35]. The core innovation of RND lies in proposing a simple quantification method for state novelty and transforming it into an optimizable intrinsic reward to facilitate agent exploration.

The technical principle of RND involves two neural networks with the same structure but different functions: a trainable main prediction network F_ϕ and a fixed target prediction network $F_{\phi'}$. Both networks are randomly initialized at the beginning of training. The target prediction network keeps its parameters unchanged during the training process and is used to generate implicit features of the environmental state s, while the main prediction network attempts to imitate the output of the target network and updates its parameters by minimizing the error between the two, that is

$$L(\phi) = ||F_\phi(s) - F_{\phi'}(s)||_2^2. \tag{25}$$

The error between the outputs of the two networks quantifies the novelty of state s, which RDN takes as an intrinsic reward.

The main prediction network constantly learns and imitates the output of the target prediction network in the visited state s. When the agent enters a familiar state (that is, the state appears frequently in the training data), the main prediction network has already learned how to predict the output of its target prediction network well with very little error, and thus the intrinsic reward obtained is also small. When an agent enters a novel or rare state (i.e., one that the main prediction network has rarely or never seen before), the main prediction network cannot accurately predict the output of its target prediction network, with a large error, and thus the intrinsic reward obtained is also large. This intrinsic reward mechanism motivates the agent to prioritize visiting and exploring new areas with large prediction errors and insufficient exploration, significantly improving the learning efficiency in sparse reward or complex exploration tasks.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

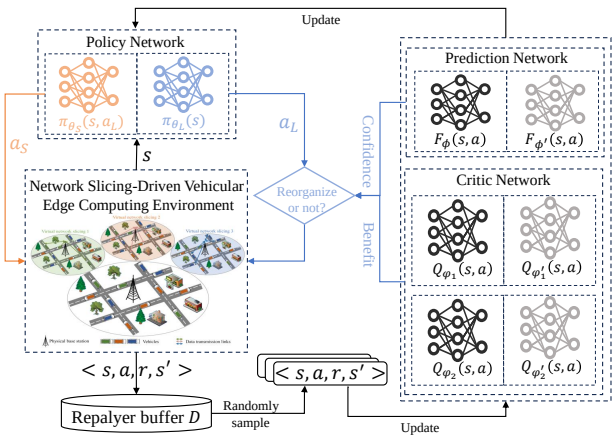


Fig. 3. The framework of CST-RL.

E. Self-Adjusting Network Slicing Scheme with RND

The network slicing schemes based on two-timescale optimization in existing studies generally preset the time window size of network slices as a fixed value, which ignores the high-speed movement characteristics of vehicles and the heterogeneity and dynamics of the types of computational tasks they carry. However, there exists a trade-off between the QoS and reconfiguration cost when utilizing a fixed time window size for network slicing. Specifically, if the time window size is set to be larger, the resource allocation and activation state of the network slice is difficult to adapt to the instantaneous dynamic demand of computational and bandwidth resources for vehicular computation tasks, which may lead to the failure of computation tasks. On the contrary, if a smaller time window size is chosen, although the flexibility of resource allocation can be improved, frequently adjusting the resource allocation and activation state of network slices will substantially increase the deployment and maintenance cost of the system. In summary, the existing two-timescale scheme based on fixed network slice time window size has significant limitations, and it is difficult to respond effectively to the complex and changing environments and task requirements described in this paper.

To address the above issue, CST-RL is proposed. As shown in Fig. 3, the core of this scheme is to realize dynamic self-adjusting of the time window size of network slicing, which can intelligently reconfigure the resource allocation and activation state of the network slice at the appropriate time according to the real-time change of the channel state as well as the actual demand of the computation task. Specifically, since the optimization problem has been modeled as a two-timescale MDP in the previous section, this means that the future state, including the resource demand of the system, is only related to the current state and not to the past state. Therefore, we utilize critic networks to implicitly predict the demand changes of each network slice in the system based on the current system state s and action a, thereby predicting the expected benefit (i.e., the Q value of a state-action pair) after taking network slice reconfiguration and non-reconfiguration actions in the current system state s. And based on the above prediction results, it guides the dynamic self-adjustment process of network slice resource allocation and activation state.

However, the utilization of critic for predictions leads to new challenges. First, the policy network is often inadequately trained in the early phase due to the lack of visited samples. In addition, as the policy improves over time, the critic may still encounter new state-action pairs. Such circumstances may lead to inaccurate predictions and unstable performance.

In general, the more frequently the critic network visits a specific state-action pair (s, a) or similar state-action pairs, the more accurate its prediction of expected returns. In view of this, we introduce the RND technique to quantify the accuracy of the critic network's prediction of state-action values on expected returns, i.e., the *confidence level* of the benefits of reconfiguring and not reconfiguring the network slices. Different from that in the previous subsection, what the RDN quantifies in the proposed scheme is the novelty of the critic network to the state-action value, that is, the inputs of the master prediction network and the target prediction network are state-action pairs (s, a). the loss function of the main prediction network in Eq. (26) can be reformulated as

$$L(\phi) = ||F_{\phi}(s,a) - F_{\phi'}(s,a)||_2^2. \tag{26}$$

It is worth noting that since the main prediction network uses the same state-action pairs for parameter updating as the critic network, the smaller the prediction error of the main prediction network for a particular state-action pair (s, a), the more frequently the critic network visits that state-action pair or similar state-action pairs, which in turn indicates that the more accurately the critic network predicts the expected return, i.e., the confidence level of predictions of system gain with or without reconfiguration are higher.

The specific steps of the proposed intelligent network slicing self-tuning scheme are as follows.

1) **Obtain actions for not reconfiguring network slices.** Actions for not reconfiguring network slices implies maintaining the current network slice resource allocation and activation state and take the corresponding short timescale actions, i.e., $a^{\text{nrec}}(t) = \{a_L^{\text{nrec}}, a_S^{\text{nrec}}\} = \{a_L(t-1), \pi_S^{\theta_S}(s(t), a_L(t-1))\}$.
2) **Predict the benefits of not reconfiguring network slices.** The critic network is utilized to predict the expected return of taking action $a^{\text{nrec}}(t)$ in the current system state $s(t)$, i.e., the benefit of no reconfiguration of the network slice $Q_{\pi}^{\varphi_i}(s(t), a^{\text{nrec}}(t))$.
3) **Calculate the confidence level of the benefits of not reconfiguring network slices.** The RND technique is then utilized to calculate the confidence level of the benefits predicted by the critic network for not reconfiguring the network slices, i.e., $C_{\text{nrec}} = e^{-\zeta||F_{\phi}(s(t),a^{\text{nrec}}(t)) - F_{\phi'}(s(t),a^{\text{nrec}}(t))||_2^2}$, where ζ is the scaling factor.
4) **Obtain actions for reconfiguring network slices.** Actions for reconfiguring network slices implies reconfiguring the resource allocation and activation state of the network slice and taking the corresponding

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

short timescale actions, i.e., $a^{\text{rec}} = \{a_L^{\text{rec}}, a_S^{\text{rec}}\} = \{\pi_L^{\theta_S}(s(t)), \pi_S^{\theta_S}(s(t), \pi_L^{\theta_S}(s(t)))\}$.

5) **Predict the benefits of reconfiguring network slices.** The critic network is utilized to predict the expected return of taking action $a^{\text{rec}}(t)$ in the current system state $s(t)$, i.e., the benefit of network slicing reconfiguration. $Q_\pi^{\varphi_i}(s(t), a^{\text{rec}}(t))$
6) **Calculate the confidence level of the benefits of reconfiguring network slices.** The RND technique is then utilized to calculate the confidence level of the benefits predicted by the critic network for reconfiguring the network slices, i.e., $C_{\text{rec}} = e^{-\zeta||F_\phi(s(t),a^{\text{rec}}(t)) - F_{\phi'}(s(t),a^{\text{rec}}(t))||_2^2}$.
7) **Execute actions based on benefits and confidence.** To facilitate exploration, we let the network slicing reconfigure as much as possible during the early phase of training when the confidence levels are low. After visiting more samples, the confidence levels of system benefits with or without reconfiguration will both increase over time. Therefore, we set the following rules: only when the confidence of both the reconfiguration gain and the non-reconfiguration gain of the network slice is greater than a threshold η and the non-reconfiguration gain of the network slice is greater than the reconfiguration gain (i.e., $(C_{\text{rec}} > \eta)\&(C_{\text{nrec}} > \eta)\&(Q_\pi^{\varphi_i}(s(t), a^{\text{nrec}}(t)) > Q_\pi^{\varphi_i}(s(t), a^{\text{rec}}(t)))$, executes the action a^{nrec}, the parameter configurations and activation states of the network slice remain unchanged; otherwise,the action a^{rec} is executed and the parameter configurations and activation states of the network slice are reconfigured.

The propsed CST-RL is shown in Algorithm 1. First, the parameters of the policy network, the critic network, and the prediction network are initialized. Subsequently, within each time slot, the system selects and executes the corresponding action $a(t)$ based on the current state $s(t)$, according to the aforementioned process, aiming at the adaptive adjustment of the network slices as well as the efficient allocation of the computational and bandwidth resources to the vehicles. Following that, the system receives the state $s(t+1)$ and the immediate reward $r(t)$ from the next time step and stores this experience tuple $\langle s(t), a(t), r(t), s(t+1)\rangle$ into the experience replay buffer D. Eventually, a batch of experience tuples set B is randomly extracted from the D and used to update the parameters of the policy network, the critic network, and the prediction network, so as to continuously optimize the network slicing and resource allocation policy.

Algorithm 1 The proposed CST-RL network slicing and resource allocation scheme.

Initialize: Initialize the parameters θ, φ_1, φ_2, ϕ, and $\phi^{'}$.
1: Let $D = \emptyset$, $\theta^{'} \leftarrow \theta$, $\varphi_i^{'} \leftarrow \varphi_i, i = 1, 2$.
2: **for** n_{epi} in $\{1, \ldots, N_{epi}\}$ **do**
3: Randomly initialize resource allocation and activation state $a_L(0)$ of network slices.
4: Obtain initial state $s(1)$ from the environment.
5: **for** t in $\{1, \ldots, T\}$ **do**
6: **for** v in $\{1, \ldots, V\}$ **do**
7: According to the type $J_v(t)$ of the task of the v-th vehicle, it is assigned to the corresponding subordinate network slice $S_*, * \in \{\text{C, D, L}\}$.
8: **end for**
9: Obtain actions a^{rec} and $a^{\text{nrec}}(t)$ for network slice reconfiguration and non-reconfiguration.
10: Predict the benefits $Q_\pi^{\varphi_i}(s(t), a^{\text{rec}}(t))$ and $Q_\pi^{\varphi_i}(s(t), a^{\text{nrec}}(t))$ for network slice reconfiguration and non-reconfiguration.
11: Calculate the confidence level C_{rec} and C_{nrec} for the benefits of network slice reconfiguration and non-reconfiguration.
12: **if** $(C_{\text{rec}} > \eta)\&(C_{\text{nrec}} > \eta)\&(Q_\pi^{\varphi_i}(s(t), a^{\text{nrec}}(t)) > Q_\pi^{\varphi_i}(s(t), a^{\text{rec}}(t)))$ **then**
13: Execute actions with noise $a(t) = a^{\text{nrec}}(t) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma_p)$ with not reconfigured network slices.
14: **else**
15: Execute actions with noise $a(t) = a^{\text{rec}}(t) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma_p)$ with reconfigured network slices.
16: **end if**
17: Observe reward $r(t)$ and state $s(t+1)$.
18: Store tuple $\langle s(t), a(t), r(t), s(t+1)\rangle$ in D.
19: Sample a random minibatch of tuples B from D.
20: Update the parameter of main critic network by minimizing $L(\varphi_i), i = 1, 2$.
21: Update the parameter of main prediction network by minimizing $L(\phi)$.
22: Update the parameter of main policy network by maximizing $J(\theta_L, \theta_S)$.
23: Update the parameter of the target network by soft updates.
24: **end for**
25: **end for**

F. Complexity analysis

The proposed network slicing and resource allocation scheme based on CST-RL consists of N_{epi} training episodes in the training phase, with each training episode containing T time slots. In each time slot, the computational complexity of the scheme mainly comprises three parts: obtaining actions, determining whether to reconfigure network slices, and updating network parameters.

The first part involves the policy network acquiring actions through forward propagation (Lines 6, 9, 13, and 15 in Algorithm 1), with a computational complexity of C_π^{f}.

The second part determines whether to reconfigure network slices by having the critic network predict the benefits of reconfiguring and not reconfiguring network slices, and using the prediction network to calculate the confidence level of the predicted benefits (Lines 7, 8, 10, and 11 in Algorithm 1), with a computational complexity of $C_{\text{RND}}^{\text{f}} + C_Q^{\text{f}}$.

The third part involves the policy network, critic network, and prediction network updating their parameters through gradient backpropagation using a batch of experience tuples set B randomly sampled from the D (Lines 20, 21, and

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

22 in Algorithm 1), with a computational complexity of $|B|(C_\pi^{\mathrm{b}} + C_Q^{\mathrm{b}} + C_{\mathrm{RND}}^{\mathrm{b}})$, where $|B|$ is size for the batch of experience tuples B, C_π^{b}, C_Q^{b}, and $C_{\mathrm{RND}}^{\mathrm{b}}$ are the computational complexities of gradient backpropagation for the three networks, respectively. Meanwhile, the target network updates its parameters through a simple copying method as shown in Equation (24) (Line 23 in Algorithm 1), and its computational complexity can be neglected.

In addition, the computational complexities of forward propagation (such as C_π^{f}, $C_{\mathrm{RND}}^{\mathrm{f}}$, and C_Q^{f}) and gradient backpropagation (such as C_π^{b}, C_Q^{b}, and $C_{\mathrm{RND}}^{\mathrm{b}}$) in deep neural networks are all related to their network structures, input and output dimensions (i.e., the dimensions of states and actions), and the number of parameters.

Therefore, the total computational complexity of the training phase is $O(N_{epi}T(C_\pi^{\mathrm{f}}+C_{\mathrm{RND}}^{\mathrm{f}}+C_Q^{\mathrm{f}}+|B|(C_\pi^{\mathrm{b}}+C_Q^{\mathrm{b}}+C_{\mathrm{RND}}^{\mathrm{b}}))$.

In addition, during the deployment stage, the inference and decision-making process of the proposed scheme consists of three parts: critic network prediction of the benefits of reconfiguring and not reconfiguring network slices, the prediction network to calculate the confidence level of prediction benefits, and the policy network obtaining actions through forward propagation. Therefore, the complexity of a single inference and decision-making in the proposed scheme is complex $O(C_\pi^{\mathrm{f}} + C_{\mathrm{RND}}^{\mathrm{f}} + C_Q^{\mathrm{f}})$.

According to Eq. (16), (17), and (18), it is known that the dimension of states is $|s| = (4L + 7)V + 9$, the dimension of long timescale actions is $|a_L| = 9$, and the dimension of short timescale actions is $|a_S| = 2V$. Therefore, the increase in state and action dimensions can negatively impact the sample efficiency and convergence rate of the CST-RL algorithm, which is a common challenge faced by deep reinforcement learning in high-dimensional practical problems. However, since the state-action space dimensionality of our algorithm grows linearly with the scale of the Internet of Vehicles (IoV), and is effectively controlled through hierarchical action decomposition and carefully designed state features, this impact remains limited without leading to the curse of dimensionality.

G. Convergence Analysis

The proposed CST-RL scheme employs a hierarchical optimization structure characterized by coupled updates for network slicing (long-timescale) and vehicle resource allocation (short-timescale). We analyze the convergence properties of this two-timescale learning process by leveraging the *Two-Timescale Stochastic Approximation (TTSA)* theory and the *Contraction Mapping* property of the Bellman operator.

1) Timescale Separation via TTSA: The stability of CST-RL can be theoretically justified via TTSA, which explicitly decouples the optimization variables into slice-level actions a_L (slow process) and vehicle-level actions a_S (fast process).

The update rules of the parameters θ_L and θ_S of the long-timescale and the short-timescale policy network form a coupled iteration system:

$$\theta_{L,k+1} = \theta_{L,k} + \eta_L \nabla J_L(\theta_{L,k}, \theta_{S,k}), \tag{27}$$

$$\theta_{S,k+1} = \theta_{S,k} + \eta_S \nabla J_S(\theta_{L,k}, \theta_{S,k}), \tag{28}$$

where η_L and η_S represent the learning rates. A key design principle is the setting of $\eta_S \gg \eta_L$, which ensures that the vehicle-level policy π_S evolves on a significantly faster timescale than π_L.

Invoking the TTSA principle, from the perspective of the slow process (θ_L), the fast process (θ_S) effectively tracks a quasi-static equilibrium $\theta_S^*(\theta_L)$. Consequently, the update of the slice-level policy approximates a gradient ascent on the asymptotic value function. Under standard stochastic approximation conditions (i.e., $\sum \eta_k = \infty, \sum \eta_k^2 < \infty$), this hierarchical separation facilitates convergence to a local optimum. This theoretical stability is empirically corroborated by the convergence curves presented in Fig. 4.

2) Critic Convergence via Contraction Mapping: The reliability of the Policy update is predicated on the accurate evaluation of Q-values by the Critic network. We demonstrate the convergence of the Critic via the contraction mapping property. We define the Bellman operator $\mathcal{B}_\pi Q(s,a)$ as:

$$\mathcal{B}_\pi Q(s,a) := \mathbb{E}_{s' \sim p(\cdot|s,a)} \left[r(s,a) + \gamma \mathbb{E}_{a' \sim \pi(\cdot|s')}[Q(s',a')] \right], \tag{29}$$

where $\gamma \in [0,1)$ is the discount factor. For any two Q-value functions Q and $\hat{Q}$, the distance under the Bellman operator satisfies the following inequality:

$$\begin{aligned} &||\mathcal{B}_\pi Q(s,a) - \mathcal{B}_\pi \hat{Q}(s,a)||_\infty \\ &= \gamma \left| \mathbb{E}\left[Q(s',a') - \hat{Q}(s',a') \right] \right| \\ &\leq \gamma \max_{s',a'} |Q(s',a') - \hat{Q}(s',a')| \\ &= \gamma ||Q - \hat{Q}||_\infty. \end{aligned} \tag{30}$$

Since $\gamma < 1$, Eq. (30) proves that the Bellman operator $\mathcal{B}_\pi$ is a γ-contraction mapping under the L_∞ norm. By the Banach Fixed-Point Theorem, starting from an arbitrary initialization, the iterative application of the Bellman operator converges to a unique fixed point Q^* (i.e., $\lim_{k\to\infty} \mathcal{B}_\pi^k Q = Q^*$). This ensures that the Critic network provides reliable gradient estimates for the Policy network, thereby stabilizing the overall CST-RL algorithm.

V. Simulation Results

In this section, the simulation setup will be presented and the effectiveness of CST-RL will be verified.

A. Simulation Setup

To validate the effectiveness of the proposed CST-RL scheme in addressing dynamic and heterogeneous computational resource allocation problems, we consider the VEC scenario illustrated as follows. In this scenario, a multi-functional base station equipped with $L = 8$ antennas is deployed on the ground, providing computational offloading services to $V = 20$ vehicles. The base station possesses a total computational capacity of 12 GHz and offers a total available bandwidth resource of 5 MHz. In addition, we adopt a discrete-time system, the entire decision cycle is partitioned into T equal time slots and each time slot is 50 ms [20]. At each time slot, vehicles generate computational tasks, which exhibit heterogeneity and dynamism, characterized by varying data sizes, computational

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

TABLE II
PROPERTIES OF THE THREE TYPES OF COMPUTATIONAL TASKS

Type of the computational task J_v	Data size of the computational task I_v	Data size of the processing result O_v	Computing resources required to process the computing task C_v	Maximum latency constraint L_v
Computational-intensive	[6000, 8000] bit	[600, 800] bit	[1500, 2500] bit/Cycle	[30, 50] ms
Data-intensive	[25000, 35000] bit	[2500, 3500] bit	[200, 400] bit/Cycle	[20, 40] ms
Latency-sensitive	[1500, 2500] bit	[150, 250] bit	[600, 1000] bit/Cycle	[3, 5] ms

complexities, and latency constraints. There are three types of computing tasks: computation-intensive, data-intensive, and latency-sensitive, and they are randomly generated by vehicles according to an equal probability distribution of 1:1:1. The detailed properties of the three types of computational tasks are shown in Table II. The value range of key properties for each task type is derived from the analysis and abstraction of typical vehicle applications (for example, autonomous driving environment perception, high-definition map update, real-time path planning, in-vehicle infotainment), and these ranges are designed to capture the fundamental differences between task types. During simulation, when an instance of a specific task type is generated, its specific parameters are uniformly sampled at random from the corresponding interval in Table II.

Within the proposed algorithmic framework, the policy network, the critic network, and the prediction network are all designed as DNNs, comprising 3-layer, 4-layer, and 4-layer fully connected structures, respectively, with each layer configured with 128 neuron units. We employed the Adam optimizer to optimize the parameters of these networks. Additionally, the exploration noise for actions is Gaussian noise with a mean of 0 and a variance of 0.1, i.e., $\mathcal{N}(0, 0.1)$. Other parameters are detailed in Table III.

TABLE III
LIST OF PARAMETERS

Parameter	Value
Size of experience replay buffer D	10^5
Discount factor γ	0.99
Learning rate of critic network	10^{-3}
Learning rate of prediction network	10^{-3}
Learning rate of policy network	5×1^{-4}
Number of training episodes N_{epi}	5000
Number of time slots per episode T	200
Soft update rate ι	0.01

To comprehensively compare the performance of the proposed CST-RL with other methods under different time scales and optimization strategies, we selected several baseline schemes for comparative analysis. Specifically, we first selected the currently widely-used two-timescale RL optimization scheme with fixed network slice time window size (i.e., FT-RL) as the baseline. In addition, the state-of-the-art single-timescale RL algorithms, Proximal Policy Optimization (PPO), Twin Delayed Deep Deterministic Policy Gradient (TD3), and heuristic Genetic Algorithms (GA) are selected for comparison, and they are regarded as the baselines of task allocation without network slicing.

Further, to deeply explore the key contributions of the CST-RL scheme, we design an ablation scheme, ST-RL, as another baseline. The ST-RL relies only on the benefits of reconfiguration vs. non-reconfiguration of the network slices predicted by the critic network to guide the dynamic reconfiguration process of the network slices, and does not introduce a confidence evaluation mechanism based on RND. With this design, we are able to clearly isolate the role that RND-based confidence plays in the CST-RL scheme and its contribution to the overall performance improvement.

B. Experiment Results

1) Impacts of Time Window Sizes of Network Slices: First, Table IV demonstrate the impact time window sizes T_r (in time slots) and reconstruction times N_r of network slice in FTRL and CST-RL on processing success rate of computational tasks R_s and system costs S_c. It can be observed in FT-RL that as the time window size of network slices increases, the system cost initially decreases (when the time window size ranges from 1 to 5), then increases (from 5 to 15), followed by another decrease (from 15 to 30), and finally increases again (from 30 to 200). Concurrently, the processing success rate of computational tasks exhibits a decreasing trend. We can also observe that CST-RL saves 27.77% of system cost compared to the scheme without network slicing (i.e., when the time window size is 200) and reduces 15.15% of system cost compared to performing constant network slicing configuration (i.e., when the time window size is 1).

This phenomenon can be primarily attributed to the following reasons: with smaller time windows, network slices can be reconfigured more frequently to accurately match the time-varying resource demands of vehicular computational tasks, thereby achieving a higher task processing success rate. As the time window size increases, the frequency of network slice reconfiguration decreases, which reduces the reconfiguration cost of network slices. However, this also leads to a decrease in the processing success rate of computational tasks and a subsequent increase in the cost of task processing. Therefore, during the process of increasing the time window size of network slices, when the reduction in reconfiguration cost of network slices outweighs the increase in task processing cost, the total system cost decreases (as seen in the phases where the time window size ranges from 1 to 5 and from 15 to 30). Conversely, when the reduction in reconfiguration cost is less than the increase in task processing cost, the total system cost increases (as observed in the phases where the time window size ranges from 5 to 15 and from 30 to 200).

Different from FT-RL, the proposed CST-RL realizes adaptive reconfiguration of network slices at the right time by introducing system gain prediction and confidence assessment mechanism. As a result, the scheme achieves an effective

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

TABLE IV
THE IMPACT TIME WINDOW SIZES T_r (IN TIME SLOTS) AND RECONSTRUCTION TIMES N_r OF NETWORK SLICE IN FTRL AND CST-RL ON PROCESSING SUCCESS RATE OF COMPUTATIONAL TASKS R_s AND SYSTEM COSTS S_c.

	FTRL													**CST-RL**
T_r	1	2	4	5	8	10	15	20	30	40	50	100	200	-
N_r	200	100	50	40	25	20	13	10	7	5	4	2	1	**9.85**
R_s	95.3%	93.2%	91.5%	91.2%	87.9%	86.1%	84.7%	81.5%	79.7%	79.2%	73.8%	66.2%	47.4%	**91.7%**
S_c	401.3	391.7	386.1	384.3	381.1	379.7	378.9	385.7	381.2	387.8	398.4	423.2	471.4	**340.5**

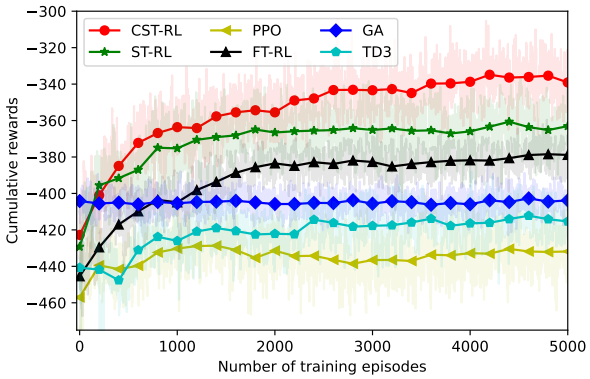


Fig. 4. The curves of the cumulative reward with the number of training episodes for CST-RL versus several baseline schemes.

trade-off and optimization between significantly reducing the system cost and maintaining a relatively stable and high computational task processing success rate.

In this section, we tested the performance of FT-RL under different fixed time window sizes, thereby determining the time window size configuration with the lowest system cost (i.e., the optimal performance). Then, the optimal time window size was used for the subsequent comparative experiments between FT-RL and the proposed scheme to ensure the rationality and persuasiveness of the experimental results.

2) Comparison of Cumulated Reward with Baselines: Fig. 4 shows the training curves of CST-RL and baselines in terms of the cumulative reward (i.e., the negative value of the system cost). Observing the results, it can be seen that after the convergence of the algorithms, the CST-RL scheme exhibits the highest cumulative reward (the lowest system cost), followed closely by the ST-RL scheme, and then by the FT-RL scheme. In contrast, GA and the two single-timescale RL schemes do not show satisfactory results, showing the necessity of two-timescale optimization.

This is mainly attributed to the fact that the single-timescale RL schemes tend to reconfigure network slices frequently, which significantly increases the overall system cost, while the FT-RL scheme is based on a two-timescale optimization framework, which effectively reduces the system cost by presetting a fixed network-slice window size and keeping the network slice resource allocation and activation state constant within each window. The ST-RL scheme goes a step further by utilizing the critic network to predict the expected benefits of reconfiguring and not reconfiguring network slices, and accordingly guides the network slices to be reconfigured at the appropriate time. Such an observation proves the effectiveness of the self-adjusting time window size for network slicing.

Moreover, the CST-RL scheme proposed in this study addresses the possible inaccuracy of the critic network in predicting the expected benefits of network slice reconfiguration by innovatively introducing the RND technique. Through this mechanism, CST-RL not only facilitates the exploration process but also optimizes the adaptive reconfiguration strategy of network slices. This improvement achieves the optimal trade-off between the efficiency of computational task execution and the cost of network slice reconfiguration, resulting in the lowest system cost level.

It is worth noting that all the schemes except GA in Fig. 4 are online learning algorithms. Their strategies will be continuously optimized with the number of training episodes, so the cumulative rewards show an upward trend. In contrast, rule-based GA is an offline learning algorithm with a fixed strategy, so its cumulative reward does not change significantly with the increase of the number of training episodes.

Furthermore, inference time is a critical factor in in-vehicle communications. To precisely evaluate the inference performance of the proposed scheme, a desktop computer equipped with a 12th Gen Intel Core i9-12900 central processing unit and the Microsoft Windows 10 operating system was utilized as the computing platform. On this platform, we executed 10,000 inference simulation tasks. Statistical analysis of the results yielded an average inference time of 0.014 milliseconds.

Fig. 5 illustrates the changes in both the number of vehicles belonging to three network slices and the size of the network slice time windows relative to the time slot number within a test episode. It can be observed that the number of vehicles within each of the three network slices dynamically fluctuates with the time slot number. This implies that the traffic load and computational resource demand for each network slice also change dynamically. As seen in the figure, when the number of vehicles within a network slice changes frequently, the size of its time window becomes smaller. This allows the resource scheduling strategy to respond more frequently to environmental changes, avoiding response delays or resource overload caused by sudden load shifts. Conversely, when the variation in the number of vehicles stabilizes, the network slice time window size increases. This reduces the reconfiguration frequency and minimizes the associated significant system cost. These results demonstrate that the proposed scheme can intelligently and dynamically select the appropriate timing for reconfiguring the activation state and resource allocation of network slices, based on these time-varying environmental conditions.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

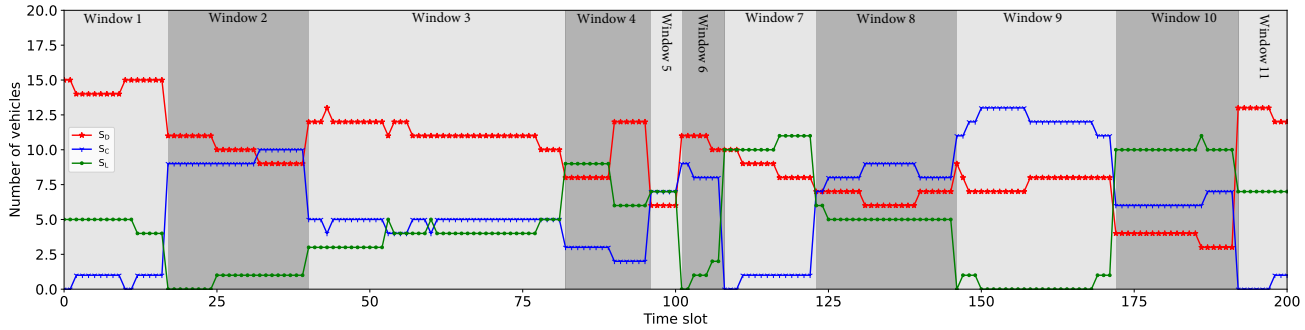


Fig. 5. The changes in both the number of vehicles belonging to three network slices and the size of the network slice time windows relative to the time slot number within a test episode

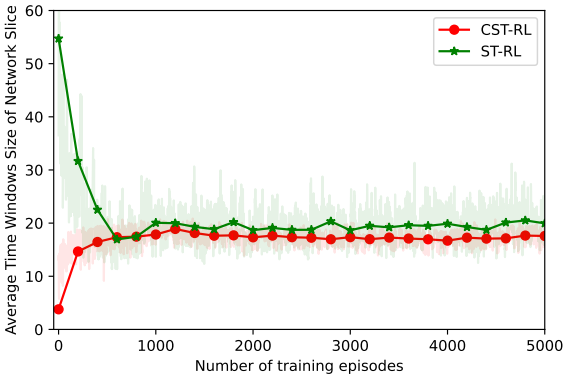


Fig. 6. The curves of the average time windows size of network slice with the number of training episodes for the CST-RL and ST-RL schemes.

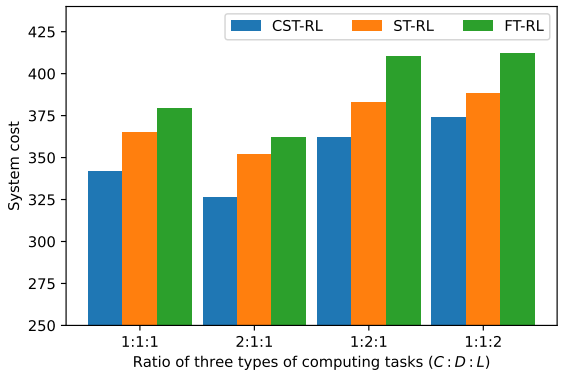


Fig. 7. Impacts of ratio of three computing task types on system cost.

3) Comparison of Time Window Size with Baselines: Several baseline algorithms (FT-RL, PPO, TD3, GA) algorithms of the time window size of the network slices of the algorithms are fixed, while those based on the CST-RL and based on the ST-RL scheme are dynamically changing. Fig. 6 shows the curves of the average time window size of network slices with the number of training episodes for the CST-RL and ST-RL schemes.

From Fig. 6, it can be seen that with the gradual increase in the number of training sets, the size of the network slicing time window of the CST-RL scheme shows a tendency to increase, while the ST-RL scheme shows a tendency to decrease, and both of them eventually keeps at a stable value. The reason for this phenomenon lies in the difference in strategies between the two schemes. For the CST-RL scheme, it employs the RDN technique to assess the confidence of the benefits. In order to facilitate the exploration process, the scheme reconfigures the network slices when the evaluated confidence is below a preset threshold. In the early stage of training, the average time window size of the network slices is relatively small because of the immature predictive ability of the CRITIC network for benefits and the generally low confidence level. As training proceeds, the system's ability to predict returns gradually increases and the confidence level improves, leading to an increase in the time window size.

4) Impacts of Ratio of Three Computing Task Types on System Cost: Fig. 7 shows how the ratio of three types of computational task $C:D:L$ (Computational-intensive, Data-intensive, and Latency-sensitive) affect the system cost. It can be seen that the three schemes have the highest system cost when the ratio of computational task types is 1:1:2, which is mainly attributed to the fact that latency-sensitive tasks need to satisfy more stringent latency constraints, leading to a significant increase in the demand of computational and communication resources. It is worth noting that the CST-RL scheme proposed in this paper always maintains the lowest system cost under multiple task ratio configuration scenarios, which further validates its superiority and robustness.

5) Impacts of Number of Vehicles on System Cost: Fig. 9 illustrates how the system cost varies with the number of vehicles under both the CST-RL approach and baseline methods. The analysis reveals that across all schemes, an increase in the number of vehicles leads to a marked rise in system costs. This is primarily attributed to the fixed total computational and bandwidth resources, which result in reduced network resources allocated per vehicle as the vehicle count grows. Consequently, task processing delays increase and may exceed predefined maximum delay constraints, substantially raising the cost of task processing failures. Moreover, the growing number of vehicles escalates system complexity, expanding the dimensionality of both state and action spaces.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

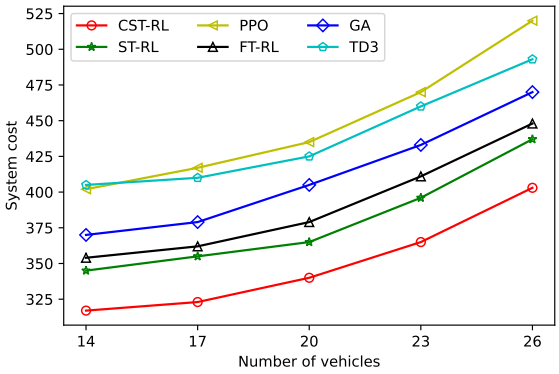
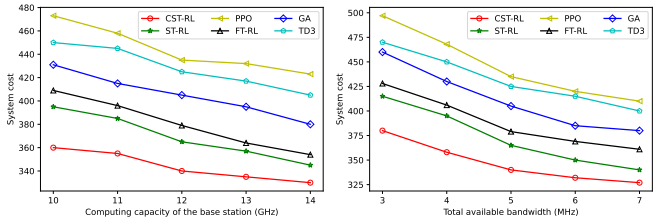


Fig. 8. Impacts of number of vehicles on system cost.

This diminishes the efficiency and accuracy of various schemes in matching heterogeneous computational task demands with network resources. Notably, compared to baseline methods, CST-RL achieves the lowest system cost under all vehicle count scenarios, highlighting its superior resource allocation capabilities and robustness.



(a) (b)

Fig. 9. Impacts of computational and communication resources on system cost. (a) Computational capacity of the base station. (b) Total available bandwidth of the base station.

6) Impacts of Computational and Communication Resources on System Cost: Fig. 9 (a) and (b) show how the system cost varies with increasing computational capacity and available bandwidth for CST-RL and baselines. It can be seen that increasing the computational capacity and bandwidth resources will lead to a decrease in system cost for all schemes. Nevertheless, the increase of such resources will also lead to higher installation costs of base stations, which is not considered in this work. Comparing different schemes, it can be seen that the proposed CST-RL has the lowest system cost under all settings. However, compared with the scenarios based on two two-timescale optimizations (ST-RL and FT-RL), the advantages of the proposed scenarios gradually decrease with the increase of the total amount of resources, showing marginal effects. Whereas, compared with the schemes based on three single time scale optimizations (PPO, TD3, and GA), the advantage of CST-RL remains significant as the total amount of resources increases. This is because the two-timescale schemes can reduce the system cost by decreasing the frequency of network slicing when the system resources are sufficient, whereas the single-timescale optimization-based schemes fail to achieve. Overall, we can conclude that the proposed CST-RL outperforms other state-of-the-art baselines.

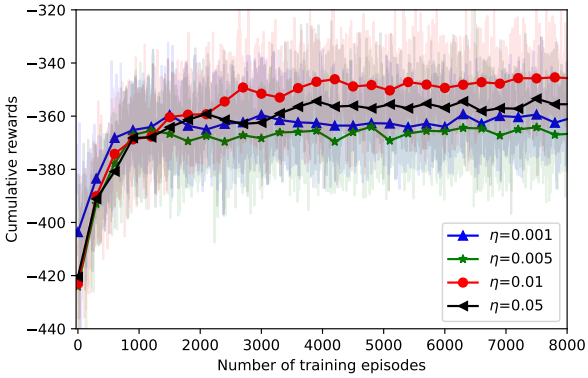


Fig. 10. The curves of the cumulative rewards with the number of training episodes for CST-RL with different confidence thresholds.

7) Hyperparameter η Sensitivity Analysis: To more quantitatively analyze the impact of the dynamic window mechanism on training convergence speed and stability, we present a sensitivity analysis of the CST-RL algorithm performance under varying confidence thresholds η in Fig. 10. The confidence threshold is set for the critic network as an indicator to decide whether the predictions of system benefits are credible or not. It can be seen that the system achieves the optimal performance, i.e., the highest cumulative reward, when the confidence threshold is 0.01. Both too-large and too-small confidence thresholds may adversely affect the performance of the scheme. Specifically, if the threshold is set too small, the scheme will exhibit the problem of under-exploration, which restricts the learning of effective strategies and may cause the system to converge to a sub-optimal strategy prematurely. On the contrary, if the threshold is set too large, the scheme will fall into the state of over-exploration, which results in a slowdown of convergence speed. Therefore, setting a reasonable confidence threshold is crucial for balancing exploration and exploitation and optimizing system performance.

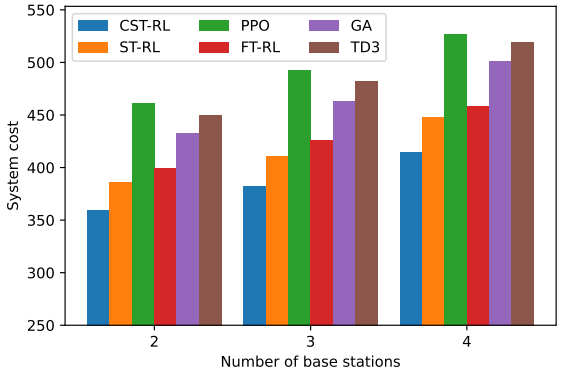


Fig. 11. The system costs of several schemes in scenarios with different numbers of base stations.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

8) Scalability Verification in Multi-Base Station Scenarios: In this section, we will verify the scalability of the proposed scheme in a multi-base station scenario. Specifically, in the multi-base station scenario, we keep the total number of vehicles and network resources (computing resources, bandwidth resources) the same as in the single-base station scenario, and only adjust the number of base stations (such as 2 base stations, 4 base stations). Moreover, considering issues such as cross-regional vehicle movement in multi-base station scenarios, the number of vehicles served by the base stations changes dynamically, which will cause the state space and action space dimensions of the reinforcement learning agent to also change accordingly. To this end, we adopt the policy network and critic network based on the attention mechanism proposed in our previous work [36] to effectively handle the dynamic changes of variable dimensions.

The simulation experiment results in Fig. 11 show that as the number of base stations increases, the system cost gradually rises. This is mainly because when the number of base stations increases, although the total amount of resources remains unchanged, the computing resources of a single base station are diluted. Meanwhile, cross-regional vehicle movement and base station switching lead to dynamic load imbalance. For instance, some base stations abandon computing requests due to task overload, while others have idle resources. This fragmentation of resources and the lag in scheduling jointly push up the overall system cost. However, in scenarios with different numbers of base stations, the system cost of the proposed scheme is the lowest, which fully demonstrates the applicability and superiority of the proposed solution in complex scenarios such as multi-base station collaboration, cross-regional vehicle movement or base station switching.

9) Optimization for System Costs with eEnergy Consumption : Our original optimization objective in problem P1 was to minimize the system cost, which includes resource consumption cost, network slice maintenance cost, network slice reconfiguration cost, and computational task processing cost, but the energy consumption of the system was not taken into account. To better align with the practical needs of battery-limited devices in vehicular networks, we have incorporated the energy consumption during computational task processing into the system cost, formulating a new optimization problem P2 to replace the original problem P1:

$$\text{P2:} \min_{\boldsymbol{B},\boldsymbol{\alpha},\boldsymbol{\beta}} \sum_{t=0}^{T} \hat{C}(t; B(t), \alpha(t), \beta(t), w^{\mathrm{I}}(t), w^{\mathrm{III}}(t)) \tag{31}$$

$$\text{s.t.} \quad (13a) - (13e), \tag{31a}$$

where $\hat{C}(t) = C(t) + \omega_{\text{ene}} C_{\text{ene}}$, ω_{ene} is the weighting factor, C_{ene} is the total energy consumption of the system for task processing, which includes the transmission energy consumed by vehicles when offloading tasks to the base station, the energy consumed by the base station servers for task processing, and the transmission energy required for the base station to return the processing results to the vehicles. This study adopts the task processing energy consumption model and task transmission energy consumption model based on the servers widely used in existing literature [36].

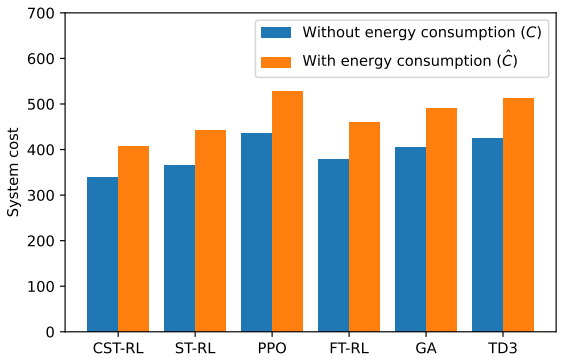


Fig. 12. The system costs of several schemes with and without energy consumption.

Fig. 12 compares the system costs, with and without energy consumption, for the proposed CST-RL-based scheme and several baseline schemes. As expected, the system costs incorporating energy consumption are significantly higher than those excluding energy consumption for all schemes. However, the proposed CST-RL-based scheme achieves the lowest system cost with energy consumption compared to the baseline schemes. This further validates its superiority and robustness across diverse scenarios and reward functions.

C. Discussion on Multiple Access Schemes

While this study employs OFDMA to ensure channel orthogonality, we acknowledge that advanced access schemes, such as Non-Orthogonal Multiple Access (NOMA) [37] and hybrid multiple access [38], can further enhance spectral efficiency. However, a distinctive advantage of our CST-RL framework is its access-agnostic nature. The proposed dual-time-scale optimization logic functions independently at both the resource partitioning (long timescale) and user association (short timescale) levels. Replacing the physical layer primarily alters the environmental feedback (the reward function derived from computational delay and energy consumption). Since the CST-RL agent learns the optimal policy π via interaction with the environment, regardless of the specific mathematical formulation of the data rate, the proposed algorithm can be readily extended to NOMA or hybrid access scenarios. This requires only updating the environment state transitions and reward calculations, without modifying the core algorithmic architecture.

VI. Conclusion

In this paper, we study the dynamic and heterogeneous resource allocation problem for VEC and introduce network slicing techniques to provide personalized services and satisfy differentiated QoS. We propose a two-timescale RL scheme, CST-RL, to minimize the system cost. CST-RL optimizes the resource allocation and activation state of network slices in the

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

long time scale in order to efficiently process computational tasks while reducing the reconfiguration cost of network slices; meanwhile, it optimizes the resource allocation strategy of vehicles in the short time scale. In addition, CST-RL utilizes the critic network in RL to predict the expected benefits from reconfiguration and non-reconfiguration of network slices, and also employs the RDN technique to evaluate the confidence level of these predicted benefits.

Experimental results show that CST-RL is able to intelligently guide the reconfiguration decisions of network slices, including whether and when to reconfigure, so as to realize the dynamic adjustment of the resource allocation and activation state of network slices. Finally, we comprehensively verify the significant advantages of CST-RL over state-of-the-art baseline schemes in terms of system cost. It is worth emphasizing that the proposed strategy is not only highly applicable to the specific VEC scenarios explored in this paper, but also provides a novel and effective approach for solving the time-window adaptive tuning challenge in a wider range of two-timescale optimization problems.

For future work, we will further investigate the problem of dynamic heterogeneous computing task offloading and resource allocation in network slicing enabled multi-base station VEC, especially focusing on the resource allocation challenges of multi-base station cooperation mechanism. In addition, in the future, we will also focus on the analysis of the impact of base station control-plane latency on system performance.

REFERENCES

[1] X. Chen, Y. Deng, H. Ding, G. Qu, H. Zhang, P. Li, and Y. Fang, "Vehicle as a service (VaaS): Leverage vehicles to build service networks and capabilities for smart cities," *IEEE Communications Surveys & Tutorials*, 2024.

[2] Z. Liu, H. Sun, G. Marine, and H. Wu, "6G IoV networks driven by RF digital twin modeling," *IEEE Transactions on Intelligent Transportation Systems*, 2023.

[3] B. Cao, Z. Sun, J. Zhang, and Y. Gu, "Resource allocation in 5G IoV architecture based on SDN and fog-cloud computing," *IEEE transactions on intelligent transportation systems*, vol. 22, no. 6, pp. 3832–3840, 2021.

[4] W. Mao, O. U. Akgul, A. Mehrabi, B. Cho, Y. Xiao, and A. Ylä-Jääski, "Data-driven capacity planning for vehicular fog computing," *IEEE Internet of Things Journal*, vol. 9, no. 15, pp. 13 179–13 194, 2022.

[5] W. Mao, O. U. Akgul, B. Cho, Y. Xiao, and A. Ylä-Jääski, "On-demand vehicular fog computing for beyond 5g networks," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 12, pp. 15 237–15 253, 2023.

[6] B. Cao, Z. Li, X. Liu, Z. Lv, and H. He, "Mobility-aware multiobjective task offloading for vehicular edge computing in digital twin environment," *IEEE Journal on Selected Areas in Communications*, 2023.

[7] Y. Hu, D. Li, P. Sun, P. Yi, and J. Wu, "Polymorphic smart network: An open, flexible and universal architecture for future heterogeneous networks," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 4, pp. 2515–2525, 2020.

[8] W. Feng, S. Lin, N. Zhang, G. Wang, B. Ai, and L. Cai, "Joint C-V2X based offloading and resource allocation in multi-tier vehicular edge computing system," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 2, pp. 432–445, 2022.

[9] W. Hamdi, C. Ksouri, H. Bulut, and M. Mosbah, "Network slicing based learning techniques for iov in 5g and beyond networks," *IEEE Communications Surveys & Tutorials*, 2024.

[10] J. Zheng, A. Banchs, and G. de Veciana, "Constrained network slicing games: Achieving service guarantees and network efficiency," *IEEE/ACM Transactions on Networking*, vol. 31, no. 6, pp. 2698–2713, 2023.

[11] M. Dai, G. Sun, H. Yu, and D. Niyato, "Maximize the long-term average revenue of network slice provider via admission control among heterogeneous slices," *IEEE/ACM Transactions on Networking*, 2023.

[12] J. Geng, Z. Qin, and S. Jin, "Dynamic resource allocation for cloud-edge collaboration offloading in VEC networks with diverse tasks," *IEEE Transactions on Intelligent Transportation Systems*, 2024.

[13] R. Huang, W. Wen, X. Chen, Z. Zhou, Q. Chen, and C. Dong, "Joint power allocation and task replication for reliability-sensitive services in NOMA-enabled vehicular edge computing," *IEEE Transactions on Vehicular Technology*, 2023.

[14] Z. Han, Y. Yang, W. Wang, L. Zhou, T. N. Nguyen, and C. Su, "Age efficient optimization in UAV-aided VEC network: A game theory viewpoint," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 12, pp. 25 287–25 296, 2022.

[15] C. Li, Y. Zhang, L. Yu, and M. Yang, "Efficient vehicle selection and resource allocation for knowledge distillation-based federated learning in UAV-assisted VEC," *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[16] L. Zhao, E. Zhang, S. Wan, A. Hawbani, A. Y. Al-Dubai, G. Min, and A. Y. Zomaya, "MESON: A mobility-aware dependent task offloading scheme for urban vehicular edge computing," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 4259–4272, 2023.

[17] L. Zhao, X. Dong, A. Hawbani, Y. Bi, Q. He, and Z. Liu, "Optimizing task offloading in VEC: A PDQKM scheme combining deep reinforcement learning and kuhn-munkres matching," *IEEE Transactions on Intelligent Transportation Systems*, 2024.

[18] H. Yonghui, J. Zuodong, Q. Peng, and T. Dan, "Task offloading and resource allocation in NOMA-VEC: A multi-agent deep graph reinforcement learning algorithm," *China Communications*, vol. 21, no. 8, pp. 79–88, 2024.

[19] G. Alkhoury, S. Berri, and A. Chorti, "Task offloading with 5G network slicing for V2X communications," in *GLOBECOM 2022-2022 IEEE Global Communications Conference*. IEEE, 2022, pp. 4425–4430.

[20] Q. Ye, W. Shi, K. Qu, H. He, W. Zhuang, and X. Shen, "Joint ran slicing and computation offloading for autonomous vehicular networks: A learning-assisted hierarchical approach," *IEEE Open Journal of Vehicular Technology*, vol. 2, pp. 272–288, 2021.

[21] H. Shen, Y. Tian, T. Wang, and G. Bai, "Slicing-based task offloading in space-air-ground integrated vehicular networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 4009–4024, 2023.

[22] S. Berri, K. Hejja, and H. Labiod, "Slicing-based offloading in vehicular edge computing," in *2021 IEEE 22nd International Conference on High Performance Switching and Routing (HPSR)*. IEEE, 2021, pp. 1–7.

[23] H. Khan, P. Luoto, S. Samarakoon, M. Bennis, and M. Latva-Aho, "Network slicing for vehicular communication," *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, p. e3652, 2021.

[24] X. Li, X. Zhang, and T. Huang, "Joint task offloading and service placement for mobile edge computing: An online two-timescale approach," *IEEE Transactions on Cloud Computing*, 2023.

[25] L. Feng, Y. Zi, W. Li, F. Zhou, P. Yu, and M. Kadoch, "Dynamic resource allocation with RAN slicing and scheduling for uRLLC and eMBB hybrid services," *Ieee Access*, vol. 8, pp. 34 538–34 551, 2020.

[26] Z. Fang, Q. Tang, R. Xie, T. Huang, T. Chen, and F. R. Yu, "Dual-timescales optimization for resource slicing and task scheduling in satellite edge computing networks," in *ICC 2024-IEEE International Conference on Communications*. IEEE, 2024, pp. 2513–2518.

[27] T. Huang, Z. Fang, Q. Tang, R. Xie, T. Chen, and F. R. Yu, "Dual-timescales optimization of task scheduling and resource slicing in satellite-terrestrial edge computing networks," *IEEE Transactions on Mobile Computing*, 2024.

[28] D. Han, Q. Ye, H. Peng, W. Wu, H. Wu, W. Liao, and X. Shen, "Two-timescale learning-based task offloading for remote IoT in integrated satellite–terrestrial networks," *IEEE Internet of Things Journal*, vol. 10, no. 12, pp. 10 131–10 145, 2023.

[29] H. Zhang and V. W. Wong, "A two-timescale approach for network slicing in C-RAN," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 6, pp. 6656–6669, 2020.

[30] W. Wu, K. Qu, P. Yang, N. Zhang, X. S. Shen, and W. Zhuang, "Cost-effective two-stage network slicing for edge-cloud orchestrated vehicular networks," in *2022 IEEE/CIC International Conference on Communications in China (ICCC)*. IEEE, 2022, pp. 968–973.

[31] X. Chen, C. Wu, Z. Zhao, Y. Xiao, S. Mao, and Y. Ji, "Hierarchical meta-reinforcement learning for resource-efficient slicing in O-RAN," in *GLOBECOM 2023-2023 IEEE Global Communications Conference*. IEEE, 2023, pp. 2729–2735.

[32] W. Liu, Y. Fu, Y. G. F. L. Wang, W. Sun, and Y. Zhang, "Two-timescale synchronization and migration for digital twin networks: A multi-agent

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3665671

deep reinforcement learning approach," *IEEE Transactions on Wireless Communications*, 2024.

[33] B. Lu, Y. Wu, L. Qian, S. Zhou, H. Zhang, and R. Lu, "Multi-agent DRL-based two-timescale resource allocation for network slicing in V2X communications," *IEEE Transactions on Network and Service Management*, 2024.

[34] X. Liu, H. Zhang, K. Long, M. Zhou, Y. Li, and H. V. Poor, "Proximal policy optimization-based transmit beamforming and phase-shift design in an IRS-aided ISAC system for the THz band," *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 7, pp. 2056–2069, 2022.

[35] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, "Exploration by random network distillation," *arXiv preprint arXiv:1810.12894*, 2018.

[36] X. Li, W. Huangfu, X. Xu, J. Huo, and K. Long, "Attention-driven marl for aoi minimization in uav-assisted intelligent transport systems," *IEEE Transactions on Intelligent Transportation Systems*, pp. 1–15, 2025.

[37] M. A. Hossain and N. Ansari, "Network slicing for NOMA-enabled edge computing," *IEEE Transactions on Cloud Computing*, vol. 11, no. 1, pp. 811–821, 2023.

[38] ——, "Hybrid multiple access for network slicing aware mobile edge computing," *IEEE Transactions on Cloud Computing*, vol. 11, no. 3, pp. 2910–2921, 2023.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 03:01:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.